MPOO · GRUPO 6 · S16 · 2026-10-12

Ahora sí, POO

Herencia: de `extends` a clases abstractas (2.1 y 2.2)

100 minutos · Java 21 · FI-UNAM · 2027-1

Ya saben escribir Java. Hoy empieza lo que hace distinta a la programación orientada a objetos.

DÓNDE VAMOS

La Unidad 1 fue la sintaxis de Java

Ya lo saben hacer	Dónde lo vimos
Tipos, casting, operadores y cadenas	S07, S09 · P3
Arreglos y referencias	S09 · P4
Selección: if, switch, enum	S10 · P5
Repetición: for, while, do-while	S12 · P6
Clases con atributos, constructor y métodos	S03, S07 · T01 · P5

Casi todo esto también existe en C. Lo que sigue, no: son las ideas que hacen distinta a la POO.

EL MAPA DEL CURSO

Los cuatro pilares de la POO

Programación orientada a objetos

Abstracción

Quedarse con lo importante del objeto

✔ ya la usan

Encapsulamiento

Proteger lo de adentro

la idea ✔ · hoy, lo mínimo

Herencia

Reutilizar: un Gerente es un Empleado

HOY

Polimorfismo

Muchas formas, una misma llamada

S20

Sintaxis de Java: tipos, selección, repetición, arreglos, clases (U1)

Hoy: un poco de encapsulamiento para poder construir la herencia completa.

EL EJEMPLO DE TODA LA CLASE

Una nómina con tres tipos de personas

	Tiene	Cómo se le paga
Empleado	nombre, sueldo base	su sueldo base
Gerente	nombre, sueldo base, **bono**	sueldo base + bono
Vendedor	nombre, sueldo base, **ventas**	sueldo base + 5 % de ventas

¿Qué tienen **en común**?
¿Qué tiene **cada uno** que los otros no?

Vamos a construir estas clases durante toda la clase y al final las tecleamos completas.

ACTO 1

La cápsula: proteger lo de adentro

⏱ ~10 min

Antes de heredar, cada clase tiene que poder cuidar sus datos.

ACTO 1 · ASÍ ESCRIBIMOS CLASES HASTA HOY

Atributos sin protección

Empleado.java

```java
class Empleado {
    String nombre;
    double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana", 20000);
        e.sueldoBase = -5000;
        System.out.println(e.sueldoBase);
    }
}
```

Consola

```
-5000.0
```

Cualquier clase puede escribir cualquier valor en el atributo, incluso un sueldo negativo.

ACTO 1 · LA CÁPSULA

private: solo la propia clase toca el atributo

Empleado.java

```
class Empleado {
    private String nombre;
    private double sueldoBase;

    Empleado(String nombre, double sueldoBase) {
        if (sueldoBase < 0) {
            sueldoBase = 0;
        }
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }

    public String getNombre() { return nombre; }
    public double getSueldoBase() { return sueldoBase; }
}
```

- **private**: el atributo solo se lee y se escribe dentro de Empleado.
- **El constructor** decide qué valores son válidos.
- **Los getters** dejan ver el valor sin poder cambiarlo.

El objeto ya se puede defender: los datos entran por el constructor y salen por los getters.

ACTO 1 · LO QUE DICE EL COMPILADOR

Desde afuera ya no se puede

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana", 20000);
        e.sueldoBase = -5000;
    }
}
```

javac

```
Main.java:4: error: sueldoBase has private access in Empleado
        e.sueldoBase = -5000;
         ^
```

ANALOGÍA

El cajero automático: ves tu saldo en la pantalla (getter), pero no abres la caja fuerte (private).

El error es una protección: javac no deja que nadie de afuera rompa el objeto.

✋ **Pregunta · acto 1**

¿Qué pasa con este main?

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Empleado e = new Empleado("Ana", -5000);
        System.out.println(e.getNombre());
        System.out.println(e.getSueldoBase());
    }
}
```

Usa la clase Empleado con private de la lámina anterior. Fíjate en el if del constructor.

a) No compila: el sueldo es negativo

b) Imprime `Ana` y `-5000.0`

c) Imprime `Ana` y `0.0`

ACTO 1 · RESPUESTA

c) El constructor protegió el dato

Empleado.java

```
    Empleado(String nombre, double sueldoBase) {
        if (sueldoBase < 0) {
            sueldoBase = 0;
        }
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
    }
```

Consola

```
Ana
0.0
```

private más constructor más getter: eso basta hoy. Setters, modificadores y paquetes llegan en la S18.

ACTO 2

El problema: código copiado

⏱ ~10 min

Escribamos Gerente y Vendedor sin herencia y veamos qué pasa.

ACTO 2 · SIN HERENCIA

Gerente y Vendedor, cada uno por su lado

Gerente.java

```java
class Gerente {
    private String nombre;
    private double sueldoBase;
    private double bono;

    Gerente(String nombre, double sueldoBase, double bono) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.bono = bono;
    }
    public String getNombre() { return nombre; }
    public double getSueldoBase() { return sueldoBase; }
}
```

Vendedor.java

```java
class Vendedor {
    private String nombre;
    private double sueldoBase;
    private double ventas;

    Vendedor(String nombre, double sueldoBase, double ventas) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.ventas = ventas;
    }
    public String getNombre() { return nombre; }
    public double getSueldoBase() { return sueldoBase; }
}
```

Las líneas resaltadas están repetidas, letra por letra, en los dos archivos.

✋ **Pregunta · acto 2**

Si cambia la regla del sueldo base, ¿cuántos archivos corriges?

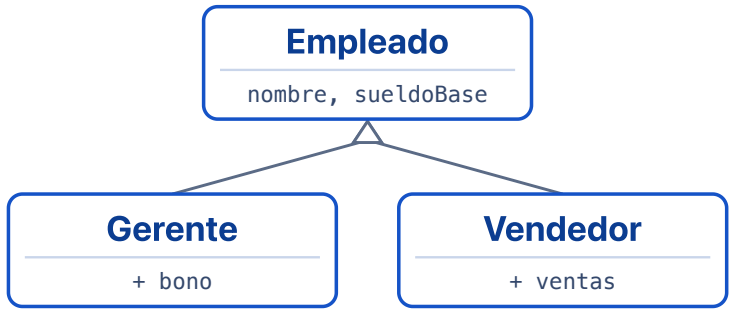
Ejemplo: el sueldo base ya no puede ser menor al salario mínimo.

Hoy son 2 clases. ¿Y si la empresa tiene Gerente, Vendedor, Cajero, Chofer y Contador?

Piensa qué pasa el día que corriges el error en cuatro archivos y se te olvida el quinto.

ACTO 2 · LA IDEA

Herencia: lo común se escribe una vez



- Un Gerente **es un** Empleado.
- Un Vendedor **es un** Empleado.
- Cada hijo agrega solo lo suyo.

ANALOGÍA

Un formulario base que todos llenan, y una hoja extra para cada puesto.

Prueba del «es un»: si la frase suena bien, la herencia tiene sentido.

✋ **Pregunta · acto 2**

¿Dónde sí tiene sentido heredar?

a) Gerente **es un** Empleado

b) Auto **es un** Motor

c) Vendedor **es una** Venta

d) Becario **es un** Empleado

Dilo en voz alta. Si la frase suena rara, no es herencia.

ACTO 2 · RESPUESTA

a y d: «es un». b y c: «tiene un»

Frase	Relación	¿Herencia?
Gerente es un Empleado	**es un**	**Sí**
Becario es un Empleado	**es un**	**Sí**
Un Auto tiene un Motor	tiene un	No: es un atributo (composición, S18)
Un Vendedor tiene ventas	tiene un	No: es un atributo

«Es un» → herencia. «Tiene un» → atributo. Composición frente a herencia lo vemos a fondo en la S18.

ACTO 3

extends: la sintaxis

⏱ ~10 min

Una palabra y Gerente recibe todo lo de Empleado.

ACTO 3 · LA SINTAXIS

class Gerente extends Empleado

Gerente.java

```
class Gerente extends Empleado {
    private double bono;
    // ...
    public double getBono() { return bono; }
}
```

Empleado	Gerente
superclase	subclase
clase padre	clase hija
clase base	clase derivada

Son tres pares de nombres para lo mismo. En el examen y en la práctica valen los tres.

ACTO 3 · QUÉ RECIBE EL HIJO

Qué se hereda y qué no

Parte de Empleado	¿La tiene el Gerente?	¿La puede usar directo?
Atributos `private` (nombre, sueldoBase)	**Sí, cada Gerente tiene su nombre**	**No: son privados de Empleado**
Métodos `public` (getNombre, getSueldoBase)	Sí	Sí
Constructores	**No: los constructores no se heredan**	**Se llaman con `super(...)`**

Lo privado viaja dentro del objeto, pero solo Empleado lo puede tocar.

ACTO 3 · EL CHOQUE

El Gerente quiere poner su nombre

Gerente.java

```
class Gerente extends Empleado {
    private double bono;

    Gerente(String nombre, double sueldoBase, double bono) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        this.bono = bono;
    }
}
```

javac

```
línea 4: constructor Empleado in class Empleado
         cannot be applied to given types;
línea 5: nombre has private access in Empleado
línea 6: sueldoBase has private access in Empleado
3 errors
```

javac da dos pistas: no puedes tocar lo privado del padre, y falta llamar a su constructor.

ACTO 4

Constructores y super()

⏱ ~15 min

Primero se construye el padre, después el hijo.

ACTO 4 · LA SOLUCIÓN

super(...) llama al constructor del padre

Gerente.java

```
class Gerente extends Empleado {
    private double bono;

    Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
    }

    public double getBono() { return bono; }
}
```

Consola

```
Ana
30000.0
8000.0
```

super(...) va en la primera línea del constructor. Empleado guarda nombre y sueldo; Gerente guarda su bono.

ACTO 4 · ORDEN DE CONSTRUCCIÓN · PASO 1 DE 4

new Gerente(...) entra al constructor de Gerente

Gerente.java

```java
class Gerente extends Empleado {
    // ...
    Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
        System.out.println("Gerente listo");
    }
}
```

Empleado.java

```java
class Empleado {
    // ...
    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        System.out.println("Empleado listo");
    }
}
```

Main.java

```java
        Gerente g = new Gerente("Ana", 30000, 8000);
```

Consola

```
```

Se pide un Gerente nuevo.

ACTO 4 · ORDEN DE CONSTRUCCIÓN · PASO 2 DE 4

La primera línea es super(...): salta a Empleado

Gerente.java

```
class Gerente extends Empleado {
    // ...
    Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
        System.out.println("Gerente listo");
    }
}
```

Empleado.java

```
class Empleado {
    // ...
    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        System.out.println("Empleado listo");
    }
}
```

Main.java

```
Gerente g = new Gerente("Ana", 30000, 8000);
```

Consola

```

```

Antes de hacer lo suyo, el hijo le pide al padre que construya su parte.

ACTO 4 · ORDEN DE CONSTRUCCIÓN · PASO 3 DE 4

Empleado termina su constructor

Gerente.java

```
class Gerente extends Empleado {
    // ...
    Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
        System.out.println("Gerente listo");
    }
}
```

Empleado.java

```
class Empleado {
    // ...
    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        System.out.println("Empleado listo");
    }
}
```

Main.java

```
        Gerente g = new Gerente("Ana", 30000, 8000);
```

Consola

```
Empleado listo
```

La parte de Empleado ya está lista: imprime primero.

ACTO 4 · ORDEN DE CONSTRUCCIÓN · PASO 4 DE 4

Regresa a Gerente y termina el suyo

Gerente.java

```java
class Gerente extends Empleado {
    // ...
    Gerente(String nombre, double sueldoBase, double bono) {
        super(nombre, sueldoBase);
        this.bono = bono;
        System.out.println("Gerente listo");
    }
}
```

Empleado.java

```java
class Empleado {
    // ...
    Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        this.sueldoBase = sueldoBase;
        System.out.println("Empleado listo");
    }
}
```

Main.java

```java
        Gerente g = new Gerente("Ana", 30000, 8000);
```

Consola

```
Empleado listo
Gerente listo
```

Como una casa: primero los cimientos (el padre), después el segundo piso (el hijo).

✋ **Pregunta · acto 4**

¿Qué imprime este main?

Vendedor.java

```
class Vendedor extends Empleado {
    // ...
    Vendedor(String nombre, double sueldoBase, double ventas) {
        super(nombre, sueldoBase);
        System.out.println("Vendedor listo");
        this.ventas = ventas;
    }
}
```

Main.java

```
        Vendedor v = new Vendedor("Hugo", 12000, 100000);
        Gerente g = new Gerente("Ana", 30000, 8000);
```

Empleado y Gerente son los de las láminas anteriores. Cada new arranca desde el padre.

a)
```
Vendedor listo
Gerente listo
```

b)
```
Empleado listo
Vendedor listo
Empleado listo
Gerente listo
```

c)
```
Vendedor listo
Empleado listo
Gerente listo
Empleado listo
```

ACTO 4 · RESPUESTA

b) Cada objeto construye primero su parte de Empleado

Consola

```
Empleado listo
Vendedor listo
Empleado listo
Gerente listo
```

Regla: en cada new, el padre termina antes que el hijo.

- Dos `new` son dos objetos: Empleado se construye dos veces.
- Aunque el println del Vendedor vaya antes que `this.ventas`, va después de super(...).
- La (c) es el error común: pensar que el hijo arranca primero.

ACTO 4 · DOS ERRORES TÍPICOS

super(...) es obligatorio y va primero

Sin super(...)

Gerente.java

```
    Gerente(String nombre, double sueldoBase, double bono) {
        this.bono = bono;
    }
```

javac

```
error: constructor Empleado in class Empleado
  cannot be applied to given types;
```

super(...) en la segunda línea

Gerente.java

```
    Gerente(String nombre, double sueldoBase, double bono) {
        this.bono = bono;
        super(nombre, sueldoBase);
    }
```

javac

```
error: call to super must be first
  statement in constructor
```

Si no escribes super(...), Java pone super() sin argumentos. Empleado no tiene ese constructor, y no compila.

ACTO 4 · DOS PALABRAS PARECIDAS

this frente a super

	this	super
Apunta a…	este objeto, completo	la parte que viene del padre
Atributo o método	`this.bono = bono;`	`super.toString()` (acto 5)
Como constructor	`this(...)`: otro constructor de la misma clase	`super(...)`: el constructor del padre
Dónde	en cualquier método o constructor	super(…) solo en la primera línea del constructor

this ya lo usaban desde la P5 para distinguir atributo de parámetro. super es su gemelo, mirando hacia arriba.

ACTO 5

Object, toString y @Override

⏱ ~15 min

Todas las clases tienen un abuelo, aunque nunca lo escriban.

ACTO 5 · ALGO RARO

Imprimimos un Gerente…

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Gerente g = new Gerente("Ana", 30000, 8000);
        System.out.println(g);
    }
}
```

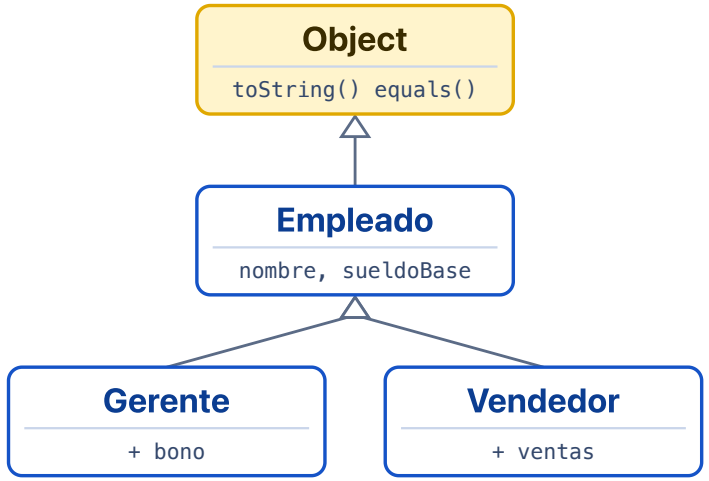
Consola

```
Gerente@6d06d69c
```

Nunca escribimos un método que imprima eso. ¿De dónde salió?

ACTO 5 · EL ABUELO DE TODOS

Toda clase hereda de Object



- Si no escribes `extends`, Java pone `extends Object`.
- Por eso todo objeto ya tiene `toString()`.
- El de Object imprime *Clase@número*: no sabe nada de nóminas.

El número después de @ cambia en cada corrida. No lo memoricen.

ACTO 5 · SOBRESCRIBIR

@Override: el hijo cambia lo que heredó

Empleado.java

```
class Empleado {
    // ...
    @Override
    public String toString() {
        return nombre + " · base " + sueldoBase;
    }
}
```

Gerente.java

```
class Gerente extends Empleado {
    // ...
    @Override
    public String toString() {
        return super.toString() + " · bono " + bono;
    }
}
```

Consola

```
Eva · base 15000.0
Ana · base 30000.0 · bono 8000.0
```

Empleado sobrescribe el toString de Object. Gerente reutiliza el de su padre con super.toString() y agrega el bono.

ACTO 5 · LA RED DE SEGURIDAD

@Override revisa que la firma sea exacta

Con @Override y tostring en minúscula

Gerente.java

```
    @Override
    public String tostring() {
        return super.toString() + " · bono " + bono;
    }
```

javac

```
error: method does not override or implement
  a method from a supertype
```

Sin @Override, mismo error de dedo

Gerente.java

```
    public String tostring() {
        return super.toString() + " · bono " + bono;
    }
```

Consola

```
Ana · base 30000.0
```

Con @Override, javac te avisa. Sin él compila, el bono desaparece y nadie te dice nada.

✋ **Pregunta · acto 5**

¿Qué hace @Override?

a) Hace que el método del padre se ejecute primero

b) Le pide a javac que confirme que estás sobrescribiendo un método heredado

c) Es obligatorio: sin él, el método no sobrescribe

Piensa en la lámina anterior: ¿qué pasó sin @Override?

ACTO 5 · RESPUESTA

b) Es una revisión, no un permiso

- La (a) es falsa: el método del padre solo corre si lo llamas con `super.metodo()`.
- La (c) es falsa: sin @Override, si la firma es exacta, sí sobrescribe. Pero si te equivocas, nadie te avisa.
- Por eso, en este curso siempre se escribe @Override.

Regla del curso: cada vez que sobrescribas, escribe @Override.

ACTO 5 · UN ADELANTO

equals también viene de Object

Main.java

```
Empleado a = new Empleado("Ana", 20000);
Empleado b = new Empleado("Ana", 20000);
System.out.println(a == b);
System.out.println(a.equals(b));
System.out.println(a.getNombre().equals(b.getNombre()));
```

Consola

```
false
false
true
```

El equals de Object compara si son el mismo objeto, como ==. Para comparar por contenido hay que sobrescribirlo: S20.

ACTO 6

Clases abstractas

⏱ ~15 min

¿Cuánto se le paga a un empleado «a secas»?

✋ **Pregunta · acto 6**

¿Cuánto se le paga a un Empleado a secas?

	calcularPago()
Gerente	sueldo base + bono
Vendedor	sueldo base + 5 % de ventas
Empleado a secas	**¿?**

En esta empresa todo empleado tiene un puesto. ¿Tiene sentido crear un Empleado que no sea ni Gerente ni Vendedor?

ACTO 6 · LA IDEA

abstract: una clase que solo sirve para heredar

Empleado.java

```java
abstract class Empleado {
    private String nombre;
    private double sueldoBase;
// ...

    public abstract double calcularPago();

// ...
}
```

- **abstract class**: no se pueden crear objetos de ella.
- **abstract double calcularPago();** no tiene cuerpo, termina en `;`.
- Es un **contrato**: cada hijo debe escribir su calcularPago().

La clase abstracta dice qué deben saber hacer todos sus hijos, pero no cómo.

ACTO 6 · LO QUE CAMBIA

new Empleado(...) ya no compila

Main.java

```
        Empleado e = new Empleado("Eva", 15000);
```

javac

```
Main.java:3: error: Empleado is abstract; cannot be instantiated
```

Ya no existe el «empleado a secas»: solo Gerentes y Vendedores.

ACTO 6 · EL CONTRATO

Un hijo que no cumple no compila

Vendedor.java

```
class Vendedor extends Empleado {
    private double ventas;

    Vendedor(String nombre, double sueldoBase, double ventas) {
        super(nombre, sueldoBase);
        this.ventas = ventas;
    }
}
```

javac

```
error: Vendedor is not abstract and does not
  override abstract method calcularPago()
  in Empleado
```

ANALOGÍA

Un contrato de trabajo: el puesto puede cambiar, pero la cláusula «calcular tu pago» la firma todo el mundo.

Al Vendedor le falta calcularPago(). javac no lo deja pasar.

ACTO 6 · CADA QUIEN CUMPLE A SU MANERA

Gerente y Vendedor implementan calcularPago()

Gerente.java

```
class Gerente extends Empleado {
    // ...
    @Override
    public double calcularPago() {
        return getSueldoBase() + bono;
    }
// ...
}
```

Vendedor.java

```
class Vendedor extends Empleado {
    // ...
    @Override
    public double calcularPago() {
        return getSueldoBase() + ventas * 0.05;
    }
// ...
}
```

Usan getSueldoBase() porque sueldoBase es privado de Empleado: la cápsula del acto 1 sigue funcionando.

✋ **Pregunta · acto 6**

¿Cuál de estas líneas compila?

a) `Empleado e = new Empleado("Eva", 15000);`

b) `Empleado e = new Gerente("Eva", 15000, 2000);`

c) `Gerente g = new Empleado("Eva", 15000);`

Usa la prueba del «es un» y recuerda que Empleado ahora es abstract.

ACTO 6 · RESPUESTA

b) Un Gerente es un Empleado

	¿Compila?	Por qué
a	No	Empleado es abstract: no se puede hacer new Empleado(...)
b	**Sí, e.calcularPago() da 17000.0**	**Un Gerente es un Empleado: cabe en una variable de tipo Empleado**
c	No	Empleado es abstract, y aunque no lo fuera, un Empleado no siempre es un Gerente

La (b) es la idea más importante de la siguiente unidad. Guárdenla para la S20.

ACTO 7

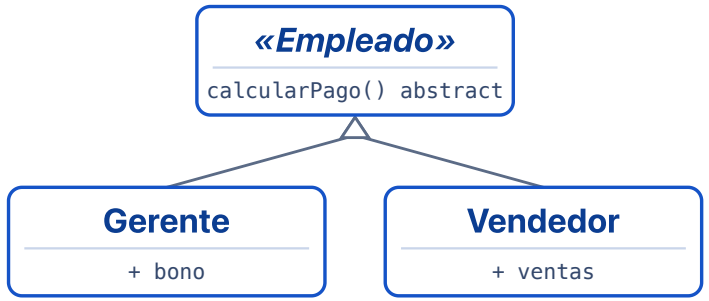
Demo en vivo

⏱ ~10 min

Tecleamos la nómina completa desde cero y la corremos.

ACTO 7 · LO QUE VAMOS A TECLEAR

La nómina completa



- Empleado: private, constructor, getters, toString y calcularPago() abstracto.
- Gerente y Vendedor: extends, super(...), @Override.
- Main: un arreglo con los dos y un for-each.

Fíjense en el orden: primero el padre, después cada hijo, al final el main.

ACTO 7 · EL MAIN

Un arreglo de Empleados, cada uno se paga a su manera

Main.java

```
public class Main {
    public static void main(String[] args) {
        Empleado[] nomina = {
            new Gerente("Ana", 30000, 8000),
            new Vendedor("Hugo", 12000, 100000)
        };
        for (Empleado e : nomina) {
            System.out.println(e + " → paga " + e.calcularPago());
        }
    }
}
```

Consola

```
Ana · base 30000.0 · bono 8000.0 → paga 38000.0
Hugo · base 12000.0 · ventas 100000.0 → paga 17000.0
```

El mismo e.calcularPago() da resultados distintos según el objeto. Eso tiene nombre: polimorfismo, en la S20.

ACTO 8

Cierre

⏱ ~10 min

Qué nos llevamos y qué sigue el miércoles.

EL MAPA, OTRA VEZ

Hoy subimos un pilar

Programación orientada a objetos

Abstracción	Encapsulamiento	Herencia	Polimorfismo
Quedarse con lo importante del objeto	Proteger lo de adentro	Reutilizar: un Gerente es un Empleado	Muchas formas, una misma llamada
✔	lo mínimo ✔ · completo en S18	✔ hoy	ya lo vimos sin nombre · S20

Sintaxis de Java: tipos, selección, repetición, arreglos, clases (U1)

Herencia: lo común se escribe una vez y cada hijo agrega solo lo suyo.

RESUMEN

Cinco ideas de hoy

Idea	En una línea
private + getter	El objeto cuida sus datos; desde afuera solo se leen
extends	El hijo recibe atributos y métodos; no recibe constructores
super(...)	Primera línea del constructor del hijo; el padre se construye primero
@Override	Cambiar lo heredado; javac revisa la firma
abstract	Clase que no se instancia; sus métodos abstractos son un contrato

Si puedes explicar cada fila con el ejemplo de la nómina, entendiste la clase.

✋ **Pregunta · salida**

Escribe la clase Director

Un Director es un Empleado. Se le paga el **doble** de su sueldo base.
¿Qué es lo **mínimo** que necesita `class Director extends Empleado` para compilar?

Dos cosas, y las dos vienen del Empleado abstracto. Escríbelo en el chat o en tu cuaderno.

PREGUNTA DE SALIDA · RESPUESTA

Un constructor con super(...) y su calcularPago()

Director.java

```java
class Director extends Empleado {
    Director(String nombre, double sueldoBase) {
        super(nombre, sueldoBase);
    }

    @Override
    public double calcularPago() {
        return getSueldoBase() * 2;
    }
}
```

Consola

```
Sofía · base 40000.0 → paga 80000.0
```

Sin super(...) no compila (acto 4). Sin calcularPago() tampoco (acto 6).

MIÉRCOLES 14 · Q007

P7 Herencia, primera parte

- Construyen **su propia jerarquía** de clases, que compile y corra.
- Con constructores encadenados con `super(...)` y `toString` sobrescrito.
- En el laboratorio y sin IA, como siempre. Entrega por push a su rama.

Lo de hoy es exactamente lo que van a teclear el miércoles. Repasen la demo.